

LAB

Propuesta

Cambio Inteligente:

Optimización con

Programación

Dinámica

DataForge

“Transformamos datos en soluciones.”





**“Transformamos
datos en
soluciones.”**

¿Quienes somos?

Somos un equipo de estudiantes apasionados por la tecnología y el desarrollo de soluciones innovadoras. En DataForge transformamos ideas y necesidades en soluciones eficientes mediante la programación, el diseño de datos y el análisis tecnológico. Nuestro objetivo es crear herramientas funcionales que generen valor y resuelvan problemas reales.

Miembros

- Maria Isabela Melendez Moreno
- Juan Jose Vergara Rojas
- Juan David Ospina Sanchez

**Errores en el cambio
→ pérdidas y clientes
insatisfechos.**



Algoritmo para Cambio de Monedas



Memoización = rapidez + exactitud.

Ejemplo en acción

Ingresa el valor del cambio: 1850

Cantidad mínima de monedas: 5

Monedas necesarias: 1000 + 500 + 200 + 100 + 50

¿Qué hay detrás?

Lenguaje: Python

Estructura: Modular por funciones

Algoritmos: Programación

Dinámica con memoización

Estructura de datos: Diccionario y listas

```
def leer_cambio():
    while True:
        try:
            cambio = int(input("Ingrese el valor del cambio: "))
            if cambio > 0:
                return cambio
            else:
                print("Valor no válido. Debe ingresar un número entero mayor que 0.")
        except ValueError:
            print("Valor no válido. Ingrese un número entero.")

def cambio_minimo_memo(monedas, valor, memo):
    if valor in memo:
        return memo[valor]
    if valor == 0:
        return 0
    min_monedas = float('inf')
    for m in monedas:
        if valor - m >= 0:
            num_monedas = 1 + cambio_minimo_memo(monedas, valor - m, memo)
            if num_monedas < min_monedas:
                min_monedas = num_monedas
    memo[valor] = min_monedas
    return min_monedas
```

```
def reconstruir_solucion(monedas, valor, memo):
    resultado = []
    while valor > 0:
        for m in monedas:
            if valor - m >= 0 and memo[valor] == 1 + memo.get(valor - m, float('inf')):
                resultado.append(m)
                valor -= m
                break
    return resultado

def main():
    monedas = [50, 100, 200, 500, 1000]
    cambio = leer_cambio()
    memo = {}
    cantidad = cambio_minimo_memo(monedas, cambio, memo)
    if cantidad == float('inf'):
        print(f"No es posible entregar un cambio exacto de {cambio} con las monedas disponibles.")
    else:
        combinacion = reconstruir_solucion(monedas, cambio, memo)
        print("Cantidad mínima de monedas:", cantidad)
        print("Monedas necesarias:", " + ".join(map(str, combinacion)), "=", cambio)

main()
```

¿POR QUÉ NUESTRA SOLUCIÓN?

Ventajas

- **Precisión**
- **Escalabilidad**
- **Experiencia del cliente mejorada**

LAB

Muchas gracias



DataForge
“Transformamos datos en soluciones.”

